PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

3ra. práctica (tipo B)
(Segundo Semestre 2026)

Duración: 1h 50 min.

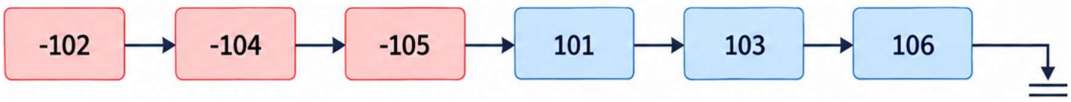
- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías **iostream, iomanip, climits, cstring, cmath o fstream. Recuerde que no puede emplear variables globales.**
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como IDE CLion.**
- Los proyectos deben llevar como nombre su código de la siguiente forma codigo_LAB3_P# (donde # representa el número de la pregunta a resolver)

Pregunta 1 (10 puntos)

Una empresa utiliza un sistema de seguridad EDR (*Endpoint Detection and Response*) para identificar equipos comprometidos dentro de su red. Los equipos se encuentran registrados en una lista simplemente enlazada, en el orden en que fueron detectados. Cada equipo se representa mediante un código entero: código positivo (equipo seguro) y código negativo (equipo comprometido).



Se solicita implementar una única **función recursiva *aislarEquipos()* de complejidad O(n)** que debe reorganizar la lista colocando primero todos los equipos comprometidos y después los seguros, retornando la cantidad de equipos comprometidos.



Se debe mantener el orden original de los equipos dentro de cada grupo. Debe reorganizar los nodos modificando únicamente sus enlaces, sin crear ni eliminar nodos ni modificar sus códigos. No puede utilizar funciones auxiliares, estructuras iterativas, arreglos, ni listas adicionales. Puede modificar la estructura Lista vista en clase considerando los punteros a nodo que necesite.

Ejemplos de ejecución:

Caso 1: Equipos comprometidos intercalados

```
Lista original:
101 -> -102 -> 103 -> -104 -> -105 -> 106

Lista reorganizada:
-102 -> -104 -> -105 -> 101 -> 103 -> 106

Equipos comprometidos: 3
```

Caso 2: Todos los equipos seguros

```
Lista original:
201 -> 202 -> 203 -> 204

Lista reorganizada:
201 -> 202 -> 203 -> 204

Equipos comprometidos: 0
```

Caso 3: Todos los equipos comprometidos

```
Lista original:
-301 -> -302 -> -303 -> -304

Lista reorganizada:
-301 -> -302 -> -303 -> -304

Equipos comprometidos: 4
```

Caso 4: Equipos comprometidos al final

```
Lista original:
401 -> 402 -> 403 -> -404 -> -405

Lista reorganizada:
-404 -> -405 -> 401 -> 402 -> 403

Equipos comprometidos: 2
```

Pregunta 2 (10 puntos)

El Laboratorio Internacional de Física de Ondas (**LIFO**) está realizando pruebas con un **acelerador lineal de partículas** con el objetivo de conocer más sobre el inicio del Universo y buscar la partícula de Dios. Durante una de las pruebas se registra, en una secuencia, la información de las partículas que ingresan al acelerador. Cada partícula se representa mediante un número entero:

- Un número **positivo** representa una partícula que se desplaza hacia la **derecha**.
- Un número **negativo** representa una partícula que se desplaza hacia la **izquierda**.
- El valor absoluto del número representa la **energía** de la partícula.

Por ejemplo: +8 representa una partícula que se desplaza hacia la derecha con energía 8, mientras que: -5 representa una partícula que se desplaza hacia la izquierda con energía 5.

Estos movimientos de las partículas originan en muchos casos colisiones entre ellas, por tal motivo se definen las reglas de colisión:

> Dos partículas solamente pueden colisionar cuando se encuentran en la siguiente situación:
> Partícula A → ← Partícula B

Es decir, una partícula positiva se encuentra a la izquierda de una partícula negativa, cuando ocurre una colisión, se tienen las siguientes situaciones:

1. Si las energías son diferentes, sobrevive la partícula de mayor energía.
2. Si ambas partículas tienen la misma energía, las dos desaparecen.
3. Una partícula que sobrevive puede provocar inmediatamente una nueva colisión con otra partícula.
4. Las partículas que se desplazan en la misma dirección nunca colisionan.

Debido a que las partículas son extremadamente **volátiles e inestables,** el sistema de adquisición de datos del acelerador solamente permite leer la secuencia **una única vez y de forma consecutiva**, desde la primera partícula hasta la última. Una vez procesada una partícula, **no es posible volver a leerla**.

A continuación, algunos ejemplos:

Ejemplo 1:

Si se cuenta con la siguiente secuencia de entrada (el orden de ingreso no se debe modificar):

12 7 -5 -15 4 -3 9 -20 6 2 -1

La respuesta serán siguientes partículas supervivientes:

2 6 -20 -15

Por ejemplo, cuando aparece -20, primero colisiona con 9 y posteriormente con 4. Sin embargo, no colisiona con -15, porque ambas partículas se desplazan hacia la izquierda.

Ejemplo 2:

Si se cuenta con la siguiente secuencia de entrada (el orden de ingreso no se debe modificar):

-5 9 27 -27 -20

La respuesta serán siguientes partículas supervivientes:

-20 -5

Se le solicita que desarrolle un programa **iterativo** que determine las partículas que sobreviven al finalizar el experimento. **Pueden mostrarse en un orden diferente al de entrada**. Los datos se ingresan en un arreglo, por lo que debe seguir el orden para su proceso. **Para el desarrollo debe emplear solo una única pila auxiliar**. El desarrollo del procesamiento de acelerador de partículas se debe realizar en una función. La complejidad temporal de su solución debe ser ***O*(n)**, donde n es la cantidad de partículas ingresadas.

Al finalizar el laboratorio, comprima cada uno de **sus proyectos** empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalos a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Jose Luis Corcuera
Rony Cueva

San Miguel, 10 de octubre del 2026